

TRƯỜNG ĐẠI HỌC
KHOA CÔNG NGHỆ THÔNG TIN

Meeting AI Agent

Hệ thống Xử lý Audio Cuộc họp và LLMOps

Môn học: Applied Large Language Models

Học kỳ: HK11 — Năm học 2025–2026

Ngày cập nhật: 13 tháng 05 năm 2026

Nhóm thực hiện

Họ và tên	Vai trò
Nguyễn Văn Minh Thiện	Trưởng nhóm, Pipeline & LLMOps

Contents

Tóm tắt	1
1 Giới thiệu	2
1.1 Bối cảnh	2
1.2 Bài toán NLP	2
1.3 Mục tiêu hệ thống	2
1.4 Phạm vi và đóng góp	3
2 Cơ sở lý thuyết và công nghệ liên quan	4
2.1 Speech-to-text và diarization	4
2.2 LLM cục bộ và structured extraction	4
2.3 RAG, guardrails và hallucination	4
2.4 LLMOps	5
3 Phương pháp	6
3.1 Pipeline xử lý	6
3.2 ASR và diarization	6
3.3 Trích xuất action items	8
3.4 Dữ liệu và feedback	8
3.5 LLMOps: dữ liệu, đánh giá và promotion	9
3.6 Observability	9
4 Triển khai hệ thống	10
4.1 Kiến trúc runtime	10
4.2 Database-backed workflow	11
4.3 Mô hình dữ liệu	11
4.4 Human feedback UI	12
4.5 MLOps services	13
4.6 API chính	14
5 Đánh giá	15
5.1 Bộ dữ liệu đánh giá	15
5.2 Kết quả định lượng	15

5.3	Cách tính metric	16
5.4	Kiểm thử vận hành	16
5.5	Đánh giá định tính	17
6	Thách thức và hạn chế	18
6.1	Diarization và speaker identity	18
6.2	Hiệu năng CPU	18
6.3	Fine-tuning phụ thuộc dữ liệu và GPU	18
6.4	Thay đổi model được kiểm soát thủ công	18
6.5	Giới hạn dữ liệu đánh giá	19
7	Hướng phát triển	20
7.1	Mở rộng dữ liệu	20
7.2	Nâng cấp evaluation	20
7.3	Nâng cao vận hành	20
7.4	Tự động hoá release có kiểm soát	20
8	Kết luận	21
	Tài liệu tham khảo	22
A	Cấu trúc dự án hiện tại	24
B	Hướng dẫn chạy	25
B.1	Chạy môi trường mặc định	25
B.2	Chạy profile MLOps	25
B.3	Deploy model promoted	25
C	Ví dụ API	26
C.1	Upload meeting	26
C.2	Lấy trạng thái meeting	26
C.3	Gửi feedback	26
D	Cấu hình môi trường quan trọng	27

List of Figures

3.1	Pipeline xử lý audio cuộc họp và vòng phản hồi của người dùng	6
3.2	Luồng xử lý WhisperX và Pyannote để tạo transcript theo lượt nói . .	7

3.3	Luồng trích xuất action items có kiểm tra schema và chuẩn hoá assignee	8
3.4	Vòng LLMOps từ feedback đến đánh giá và promotion model	9
4.1	Kiến trúc runtime của Meeting AI Agent	10
4.2	Mô hình dữ liệu chính của Meeting AI Agent	11
4.3	Luồng human feedback từ transcript evidence đến dữ liệu cải tiến . . .	13
5.1	Kết quả baseline của Qwen2.5-3B trên smoke test có nhãn	16

List of Tables

5.1	Kết quả baseline trên smoke test	15
5.2	Kết quả theo từng sample	15

Tóm tắt

Báo cáo này trình bày **Meeting AI Agent**, một hệ thống xử lý audio cuộc họp theo hướng end-to-end: nhận file âm thanh, tạo transcript có phân tách lượt nói, trích xuất action items bằng mô hình ngôn ngữ lớn, cho phép người dùng hiệu chỉnh kết quả và lưu các hiệu chỉnh đó làm dữ liệu cải tiến hệ thống. Kiến trúc hiện tại gồm Next.js frontend, FastAPI backend, Celery worker, Redis broker/cache, PostgreSQL làm nguồn dữ liệu chính, Ollama phục vụ Qwen2.5-3B, cùng Prometheus/Grafana cho quan sát trạng thái.

Pipeline chính bao gồm ingest audio, chuẩn hoá âm thanh, WhisperX ASR, speaker diarization bằng WhisperX/Pyannote, lưu transcript theo từng lượt nói, gọi LLM với prompt có guardrails, chuẩn hoá assignee theo roster, human feedback, đồng bộ lịch và lưu artifact để truy vết. Phần LLMOps tập trung vào dữ liệu feedback, kiểm tra schema, đánh giá hồi quy, drift tracking và quy trình promotion có kiểm soát. Fine-tuning được thiết kế như một bước mở rộng khi có GPU và đủ dữ liệu correction; kết quả báo cáo hiện tại tập trung vào pipeline baseline và khả năng đo lường.

Kết quả đánh giá hiện tại trên bộ smoke test `data/eval/gold_smoke.jsonl` gồm 5 mẫu và 13 action items cho model nền Qwen2.5-3B: precision 0.767, recall 0.700, F1 0.727 và schema failure rate 0.0%. Các số liệu này là baseline trước fine-tuning; chúng cho thấy pipeline đã hoạt động ổn định về định dạng và có cơ sở để tiếp tục cải thiện chất lượng trích xuất bằng dữ liệu thật.

Chapter 1

Giới thiệu

1.1 Bối cảnh

Sau mỗi cuộc họp, thông tin quan trọng thường bị phân tán trong ghi chú thủ công, tin nhắn follow-up hoặc trí nhớ của từng người tham gia. Cách xử lý này làm action items dễ bị bỏ sót, deadline không rõ ràng và người phụ trách có thể bị gán sai. Dự án **Meeting AI Agent** được xây dựng nhằm chuyển audio cuộc họp thành dữ liệu có cấu trúc: transcript theo lượt nói, danh sách công việc, người phụ trách, hạn chót, mức ưu tiên và bảng chứng transcript liên quan.

Trọng tâm của dự án không chỉ là gọi một LLM để tóm tắt nội dung. Hệ thống được thiết kế như một pipeline xử lý thông tin có thể truy vết: audio được chuẩn hoá, transcript được lưu theo lượt nói, kết quả LLM được kiểm tra bằng schema, người dùng có thể hiệu chỉnh và các hiệu chỉnh được lưu lại cho đánh giá hoặc huấn luyện tiếp theo. Cách tiếp cận này phù hợp với bài toán Applied LLM, nơi chất lượng hệ thống phụ thuộc đồng thời vào mô hình, dữ liệu, prompt, kiểm chứng kết quả và vòng lặp phản hồi.

1.2 Bài toán NLP

Dự án kết hợp ba nhóm bài toán chính:

- **ASR**: chuyển audio thành transcript bằng WhisperX.
- **Speaker diarization**: tách và gán nhãn các lượt nói bằng WhisperX/Pyannote. Speaker mặc định là các nhãn ẩn danh như `SPEAKER_00`; người dùng có thể map sang tên thật khi review.
- **Information extraction**: dùng Qwen2.5-3B qua Ollama để trích xuất action items theo JSON schema, sau đó chuẩn hoá assignee theo roster của cuộc họp.

1.3 Mục tiêu hệ thống

Các mục tiêu chính của hệ thống được xác định như sau:

1. Xử lý audio cuộc họp end-to-end qua giao diện web.
2. Lưu transcript, speaker turns, tasks, feedback và artifacts vào database để audit.
3. Cho phép human-in-the-loop sửa speaker mapping và task assignment dựa trên bằng chứng transcript.
4. Cung cấp dashboard Prometheus/Grafana cho trạng thái pipeline, worker, task và feedback.
5. Chuẩn bị dữ liệu và quy trình đánh giá để có thể mở rộng sang fine-tuning khi đủ correction.

1.4 Phạm vi và đóng góp

Phiên bản hiện tại đã triển khai được pipeline end-to-end và có thể vận hành trong môi trường Docker Compose của dự án. Báo cáo tập trung vào đóng góp kỹ thuật cốt lõi: kiến trúc xử lý audio-LLM, cơ chế lưu vết dữ liệu trong database, giao diện human feedback, đánh giá baseline và nền tảng LLMOps cho vòng lặp cải tiến. Các nội dung như mở rộng quy mô, chính sách retention dài hạn, canary release hoặc quản trị hạ tầng nhiều tenant được xem là phần nâng cao, không phải điều kiện để hệ thống hiện tại hoạt động.

Chapter 2

Cơ sở lý thuyết và công nghệ liên quan

2.1 Speech-to-text và diarization

Whisper [7] chứng minh hiệu quả của mô hình ASR được huấn luyện trên dữ liệu âm thanh quy mô lớn. WhisperX [2] mở rộng Whisper bằng alignment theo word-level timestamp và tích hợp tốt hơn với diarization. Đối với bài toán cuộc họp, timestamp và lượt nói là thông tin quan trọng vì chúng cho phép hệ thống liên kết một action item với đoạn hội thoại nguồn, thay vì chỉ lưu một bản transcript dạng plain text.

Speaker diarization được xử lý bằng Pyannote [3]. Với audio cuộc họp có nhiều người nói, diarization là điều kiện quan trọng để UI hiển thị bằng chứng theo speaker và giúp người dùng xác nhận ai nói nội dung nào. Hệ thống không giả định một file audio chỉ có một người nói.

2.2 LLM cục bộ và structured extraction

Qwen2.5 [4] được chọn vì có chất lượng instruction-following tốt trong kích thước nhỏ, phù hợp chạy cục bộ qua Ollama. Thay vì yêu cầu model sinh văn bản tự do, pipeline yêu cầu output theo JSON schema và kiểm tra lại bằng guardrails. Cách tiếp cận này giảm lỗi định dạng, giúp backend lưu action items vào database nhất quán và tạo điều kiện thuận lợi cho việc đánh giá định lượng.

2.3 RAG, guardrails và hallucination

FAISS [8] thường được dùng để truy hồi ngữ cảnh liên quan. Trong dự án, RAG/context enrichment hỗ trợ LLM xử lý transcript dài và đưa thêm evidence turn khi cần. Guardrails được dùng để giảm rủi ro prompt injection, PII leakage, lỗi schema và hallucination, phù hợp với các rủi ro đã được khảo sát trong nghiên cứu về hallucination [10].

2.4 LLMOps

MLOps/LLMOps nhấn mạnh việc đưa model vào vòng đời có đo lường, versioning, feedback và kiểm soát thay đổi [9]. Với LLM, vấn đề không chỉ là training mà còn là chất lượng prompt, drift, dữ liệu correction, regression evaluation và khả năng so sánh giữa các phiên bản. Vì vậy dự án tách rõ ba bước: thu thập feedback, đánh giá/fine-tune khi đủ điều kiện, và promotion có audit.

Chapter 3

Phương pháp

3.1 Pipeline xử lý

Pipeline xử lý được thiết kế theo dạng nhiều bước, trong đó mỗi bước tạo ra một artifact hoặc bản ghi có thể kiểm tra lại. Hình 3.1 mô tả luồng xử lý chính từ lúc người dùng upload audio đến khi hệ thống tạo action items và ghi nhận feedback.

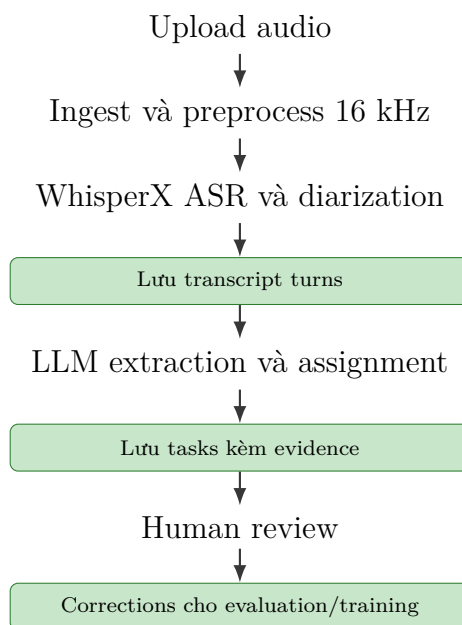


Figure 3.1: Pipeline xử lý audio cuộc họp và vòng phản hồi của người dùng

Backend lưu trạng thái cuộc họp và kết quả trung gian vào PostgreSQL. Thiết kế này giúp worker có thể restart mà không mất toàn bộ tiến độ, frontend có thể resume job, và dữ liệu feedback có thể được xuất lại cho đánh giá hoặc huấn luyện.

3.2 ASR và diarization

Module STT chịu trách nhiệm gọi WhisperX và diarization. Kết quả diarization được lưu dạng raw artifact và được dùng để gán speaker cho từng segment/turn. Hệ thống

truyền waveform trực tiếp cho diarization để tránh phụ thuộc vào decode nội bộ khi thư viện audio runtime thiếu `torchcodec` đầy đủ.

Trên UI, speaker ban đầu vẫn là nhãn kỹ thuật như `SPEAKER_00`. Người dùng xem transcript evidence và gán speaker này cho participant phù hợp. Thiết kế này đúng với bản chất audio một cuộc họp có thể có nhiều speaker, không ép toàn bộ audio về một người nói duy nhất.

Luồng WhisperX trong hệ thống gồm bốn bước tách biệt. Thứ nhất, audio đã được preprocess về WAV mono 16 kHz và được nạp vào pipeline. Thứ hai, model WhisperX chạy ASR để tạo các segment văn bản kèm timestamp thô. Thứ ba, alignment model căn chỉnh lại timestamp ở mức word-level, giúp hệ thống có căn cứ thời gian tốt hơn khi chia transcript theo lượt nói. Thứ tư, nếu bật diarization, Pyannote tạo các đoạn speaker; sau đó WhisperX gán speaker vào từng word/segment. Khi segment không có speaker trực tiếp, code lấy speaker xuất hiện nhiều nhất trong danh sách word làm fallback; nếu vẫn không có thì dùng `SPEAKER_UNKNOWN`.

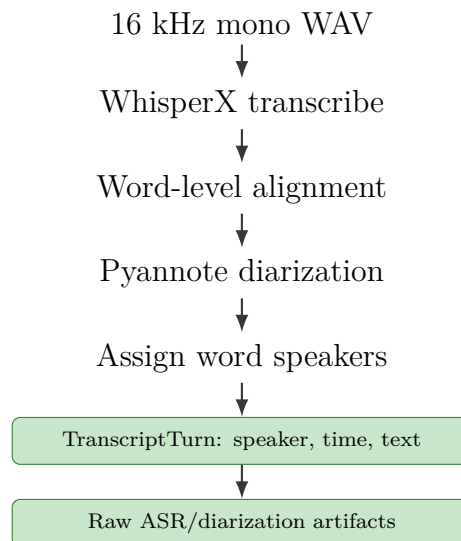


Figure 3.2: Luồng xử lý WhisperX và Pyannote để tạo transcript theo lượt nói

Đầu ra chuẩn hoá của bước này là danh sách `TranscriptTurn`, gồm `turn_id`, `speaker_id`, `start_ms`, `end_ms`, `text` và `asr_confidence`. Đây là đơn vị dữ liệu trung tâm cho các bước sau: prompt LLM được dựng từ danh sách turn, action item lưu `source_turn_ids`, UI hiển thị evidence theo turn, và feedback có thể truy ngược về đoạn hội thoại nguồn.

3.3 Trích xuất action items

LLM được gọi qua Ollama với model cấu hình mặc định Qwen2.5-3B. Prompt yêu cầu output là JSON array, chỉ được trích xuất task có bằng chứng trong transcript và assignee phải thuộc roster hoặc được đánh dấu cần review. Sau khi model trả kết quả, backend chạy validation và assignment normalization trước khi lưu.

Các task lưu trong database có thể kèm `source_turn_ids`. UI dùng các turn này để hiển thị bằng chứng cho người review, giúp việc sửa assignee hoặc mô tả task không chỉ dựa vào cảm tính.

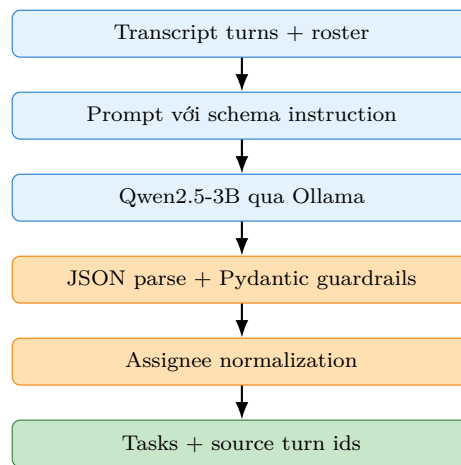


Figure 3.3: Luồng trích xuất action items có kiểm tra schema và chuẩn hoá assignee

3.4 Dữ liệu và feedback

Nguồn dữ liệu hiện tại gồm:

- Gold smoke set: bộ đánh giá nhỏ gồm 5 mẫu meeting và 13 action items.
- Synthetic training set: dữ liệu synthetic dùng làm bootstrap.
- Collected training set: dữ liệu export/thu thập cho training.
- File feedback corrections JSONL: artifact được export từ bảng feedback trong database.

Database là nguồn dữ liệu chính cho feedback. JSONL không phải nơi hệ thống ghi trực tiếp mỗi lần người dùng sửa, mà là artifact training được export để dùng với HuggingFace/TRL/Unsloth.

3.5 LLMOps: dữ liệu, đánh giá và promotion

Luồng LLMOps của dự án không chỉ là một câu lệnh fine-tune. Nó gồm các bước có đầu vào và đầu ra rõ ràng: thu thập correction từ người dùng, export correction thành JSONL huấn luyện, kiểm tra contract dữ liệu, chạy đánh giá trên gold set, ghi kết quả/promotion manifest và chỉ chuyển model serving khi kết quả được chấp nhận.

Khi số correction mới đạt ngưỡng retrain (hiện cấu hình 50), Celery Beat hoặc REST API có thể trigger retrain check. Hàm retrain không tự tạo dữ liệu giả nếu DB không export được feedback; thay vào đó nó dừng với lý do rõ ràng. Dữ liệu training được kiểm tra bằng module validate trước khi chạy training. Fine-tuning dùng QLoRA/LoRA [5, 6] qua dependency nhóm train; worker train chạy trên queue riêng, tách khỏi worker xử lý meeting bình thường.

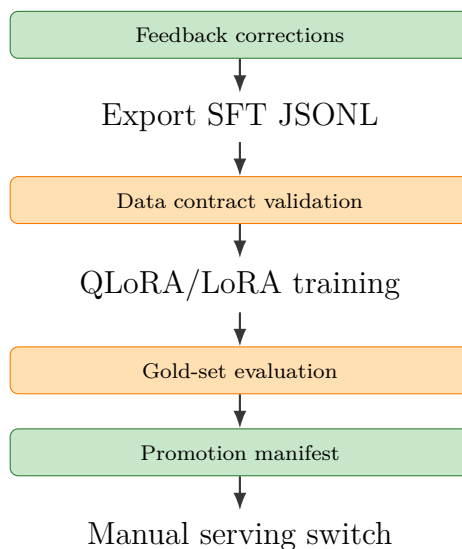


Figure 3.4: Vòng LLMOps từ feedback đến đánh giá và promotion model

Sau khi có model candidate, hệ thống chạy evaluation và ghi promotion manifest gồm model tag, output directory, checksum artifact, MLflow metadata nếu có, eval result và thông tin rollback. Việc chuyển model serving được giữ ở dạng thao tác có chủ đích để kết quả đánh giá luôn được kiểm tra trước khi thay đổi mô hình đang phục vụ.

3.6 Observability

Prometheus thu metrics từ API/worker, Grafana hiển thị trạng thái pipeline, task, worker, feedback, drift và MLOps. Các số liệu quan trọng được tính từ database khi cần để tránh mất trạng thái sau restart. Dashboard hiện phục vụ mục tiêu quan sát hệ thống trong phạm vi dự án; các chỉ số SRE chuyên sâu có thể được bổ sung khi tải xử lý và yêu cầu vận hành tăng lên.

Chapter 4

Triển khai hệ thống

4.1 Kiến trúc runtime

Hệ thống được tách thành các thành phần có trách nhiệm rõ ràng: frontend nhận thao tác người dùng, API điều phối request và truy cập dữ liệu, worker xử lý các tác vụ tốn thời gian, còn các dịch vụ nền đảm nhiệm hàng đợi, lưu trữ, inference và quan sát trạng thái. Hình 4.1 mô tả kiến trúc runtime hiện tại.

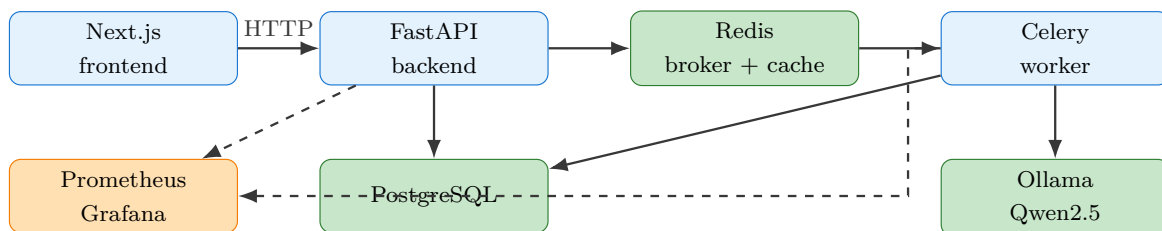


Figure 4.1: Kiến trúc runtime của Meeting AI Agent

Các thành phần chính gồm:

- **Frontend:** thư mục `web/`, xây bằng Next.js. UI hỗ trợ upload audio, theo dõi trạng thái job, review transcript, speaker mapping, task feedback và roster/history.
- **API:** `src/meeting_agent/api/main.py`, cung cấp REST endpoints cho meeting, transcript, feedback, metrics, retrain và deploy.
- **Worker:** Celery worker xử lý pipeline audio và routing queue.
- **Storage:** PostgreSQL lưu meeting, participants, transcript turns, tasks, artifacts, feedback corrections, calendar events và worker registry.
- **Queue/cache:** Redis dùng cho Celery broker và cache một số bước inference.
- **LLM serving:** Ollama phục vụ Qwen2.5-3B và embedding model nếu được cấu hình.
- **Monitoring:** Prometheus và Grafana để theo dõi API, worker, pipeline và feedback.

4.2 Database-backed workflow

Database đóng vai trò là nguồn dữ liệu trung tâm của hệ thống. Các trạng thái quan trọng được lưu trong PostgreSQL thay vì chỉ nằm trong file artifact cục bộ:

- Transcript turns thay vì chỉ nằm trong file artifact.
- Meeting participants và speaker mapping.
- Action items và feedback corrections.
- Calendar events để audit việc đồng bộ lịch.
- Worker registry và metrics phục vụ dashboard.
- Raw artifacts như diarization output và model/prompt metadata.

Thiết kế này giúp hệ thống dễ truy vết hơn và là nền tảng cho feedback export/fine-tuning. Khi một task được người dùng sửa, hệ thống vẫn có thể truy ngược về transcript turn, meeting, model metadata và correction tương ứng.

4.3 Mô hình dữ liệu

Mô hình dữ liệu được thiết kế xoay quanh thực thể **Meeting**. Một meeting là đơn vị xử lý chính, được tạo ngay khi người dùng upload audio và được cập nhật dần qua các trạng thái **pending**, **processing**, **completed** hoặc **failed**. Các bảng con lưu transcript, task, participant, feedback, artifact và calendar event đều tham chiếu về **meeting_id**. Cách tổ chức này giúp truy vấn theo meeting đơn giản, đồng thời hỗ trợ audit toàn bộ vòng đời xử lý của một audio.

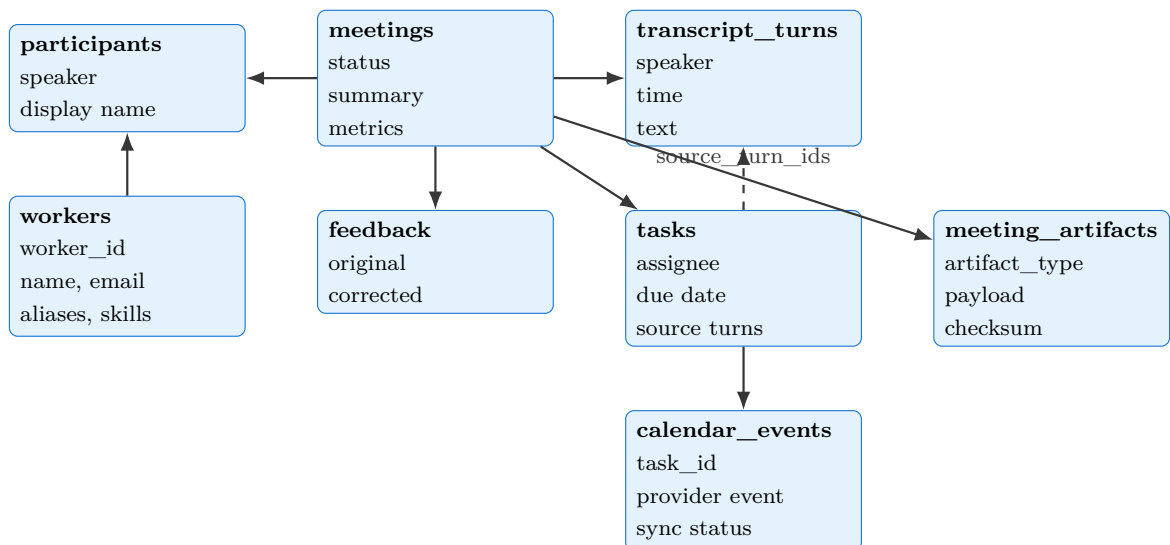


Figure 4.2: Mô hình dữ liệu chính của Meeting AI Agent

Bảng	Vai trò	Trường đáng chú ý
meetings	Bản ghi gốc của một lần upload và xử lý audio.	status, duration_ms, summary_text, run_metrics, model_version.
transcript_turns	Transcript đã chuẩn hoá theo lượt nói, dùng cho UI, prompt và training export.	turn_id, speaker_id, speaker_name, start_ms, end_ms, asr_confidence.
tasks	Action items sau khi qua LLM, guardrails và assignment.	description, assignee_id, due_date, priority, status, bucket, source_turn_ids.
feedback_correction	Điều chỉnh của người dùng, là dữ liệu chính cho vòng LLMOps.	Cặp trường gốc/sửa như description, assignee, due date; kèm cờ false positive và missing.
meeting_participants	Mapping giữa speaker diarization và participant trong meeting.	speaker_id, display_name, worker_id.
workers	Roster người tham gia/người có thể được gán task.	worker_id, name, aliases, skills.
meeting_artifacts	Lưu artifact thô hoặc dẫn xuất để audit.	artifact_type, payload, checksum, artifact_metadata.
calendar_events	Theo dõi việc đồng bộ action item sang calendar provider.	provider, provider_event_id, status, last_error.

Điểm quan trọng trong mô hình này là `tasks.source_turn_ids`. Thay vì chỉ lưu mô tả task, mỗi action item giữ danh sách turn nguồn. Nhờ đó UI có thể hiển thị bằng chứng, feedback có thể chỉ ra task nào sai, và evaluation/training export có thể gán correction với ngữ cảnh hội thoại ban đầu.

4.4 Human feedback UI

Bước Human Feedback đã được điều chỉnh theo đúng bản chất multi-speaker:

- Panel transcript hiển thị lượt nói theo speaker.
- Panel speaker evidence cho thấy mỗi `SPEAKER_xx` đã nói những đoạn nào.
- Người dùng map speaker ẩn danh sang participant thật thay vì hệ thống tự áp một speaker cho toàn bộ audio.
- Task card hiển thị evidence turn để người dùng biết tại sao task được trích xuất hoặc gán cho người đó.
- Layout review responsive hơn, không bị co lại thành một cụm nhỏ ở giữa màn hình.

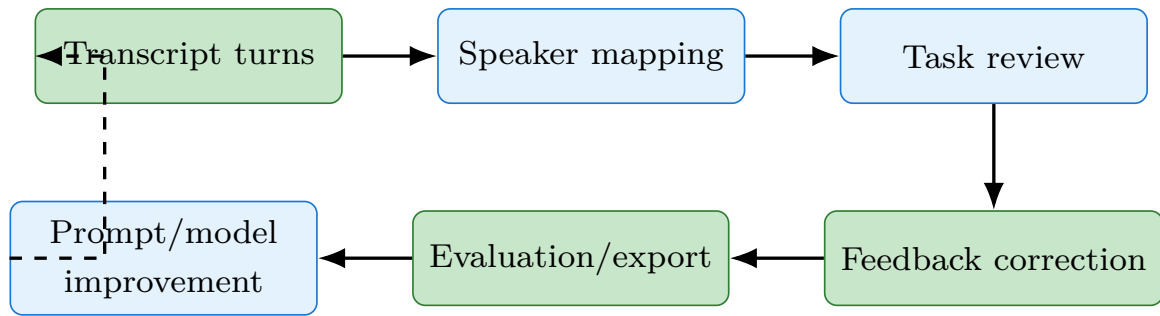


Figure 4.3: Luồng human feedback từ transcript evidence đến dữ liệu cải tiến

4.5 MLOps services

Profile `mlops` trong Docker Compose bổ sung các thành phần dành cho đánh giá và huấn luyện mở rộng:

- `beat`: Celery Beat schedule cho retrain check và drift check.
- `trainer`: worker riêng chỉ consume queue `mlops`.
- `mlflow`: tracking server cho experiment khi chạy training.
- `Dockerfile.train`: image tách riêng dependency GPU/training khỏi runtime API.

Các lệnh chính khi cần chạy nhánh MLOps:

```

1 make mlops-up
2 make retrain-check
3 make retrain-force
4 make train-image
5 make deploy-promoted-model APPLY=1
  
```

Listing 4.1: Lệnh vận hành MLOps

4.6 API chính

Endpoint	Mục đích	Ghi chú
POST /meetings	Upload audio	Tạo meeting job
GET /meetings/{id}	Xem trạng thái	Frontend
GET /meetings/{id}/transcript	Lấy transcript turns	polling/resume Dùng cho review
POST /meetings/{id}/feedback	Gửi correction	Lưu DB
POST /admin/retrain	Trigger retrain	Đưa vào queue mlops
POST /admin/deploy-promoted-model	Chuyển sang model promoted	Cần thao tác có chủ đích
GET /metrics	Prometheus metrics	Monitoring

Chapter 5

Đánh giá

5.1 Bộ dữ liệu đánh giá

Đánh giá hiện tại dùng `data/eval/gold_smoke.jsonl`, gồm 5 mẫu meeting được annotate thủ công với tổng cộng 13 action items. Bộ dữ liệu này được dùng như một smoke test có nhãn để kiểm tra regression, schema và chất lượng trích xuất ở mức baseline. Do kích thước còn nhỏ, kết quả chưa được diễn giải như một benchmark đại diện cho mọi loại cuộc họp.

Model được đánh giá là Qwen2.5-3B base chạy qua Ollama, chưa fine-tune. Vì vậy kết quả bên dưới là baseline trước khi sử dụng dữ liệu feedback thật để cải thiện mô hình hoặc prompt.

5.2 Kết quả định lượng

Table 5.1: Kết quả baseline trên smoke test

Metric	Kết quả	Mục tiêu
Precision	0.767	≥ 0.85
Recall	0.700	≥ 0.90
F1	0.727	–
Schema failure rate	0.0%	$\leq 5.0\%$

Table 5.2: Kết quả theo từng sample

Sample	Gold	Precision	Recall	F1
0	2	0.50	0.50	0.50
1	3	0.67	0.67	0.67
2	3	0.67	0.67	0.67
3	2	1.00	1.00	1.00
4	3	1.00	0.67	0.80

Kết quả cho thấy schema đã ổn định, nhưng recall còn thấp so với mục tiêu. Điều này phù hợp với trạng thái hiện tại: hệ thống đã có pipeline và đo lường, còn chất lượng model cần cải thiện bằng dữ liệu feedback/fine-tuning.

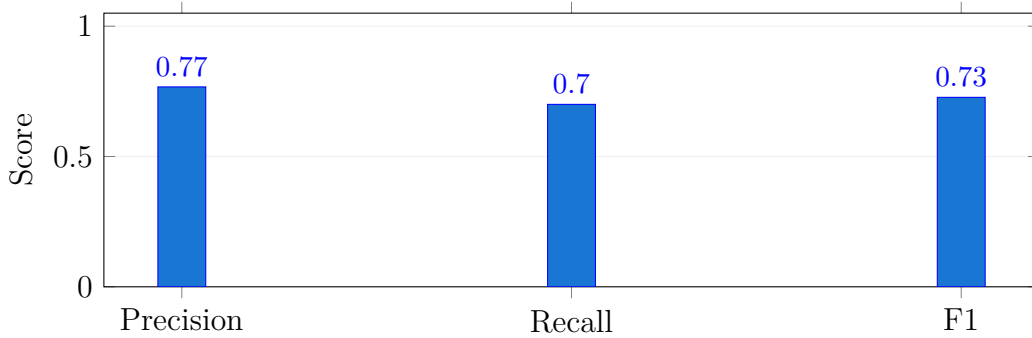


Figure 5.1: Kết quả baseline của Qwen2.5-3B trên smoke test có nhãn

5.3 Cách tính metric

Evaluation harness đo chất lượng extraction ở mức action item. Mỗi sample gồm transcript turns, roster và danh sách action items gold. Hệ thống chạy lại hàm extraction, chuyển kết quả Pydantic sang JSON, sau đó so sánh từng predicted task với gold task.

Matching mô tả task dùng BM25 trên trường **description**; nếu điểm BM25 không vượt ngưỡng, code fallback sang Jaccard overlap để tránh bỏ sót các câu ngắn có từ vựng trùng rõ ràng. Với mỗi predicted task, nếu match với một gold task chưa được dùng, hệ thống tính là true positive; predicted task không match là false positive; gold task không được match là false negative. Precision, recall và F1 được tính từ các giá trị này. Assignee accuracy chỉ được tính trên các true positive có cả assignee dự đoán và assignee gold.

Schema failure rate đo số sample mà pipeline không trả được output hợp lệ, ví dụ lỗi parse JSON hoặc lỗi guardrail nghiêm trọng làm extraction thất bại. Hallucination trong evaluation được ước lượng bằng overlap giữa từ nội dung trong task description và transcript; nếu overlap thấp hơn ngưỡng, predicted task bị xem là không có bằng chứng đủ mạnh trong transcript. Các metric này chưa thay thế đánh giá thủ công, nhưng đủ để phát hiện regression khi thay prompt, model hoặc dữ liệu training.

5.4 Kiểm thử vận hành

Các kiểm tra đã được dùng để xác nhận hành vi hệ thống:

- Ruff check cho mã Python.

- Pytest cho các phần STT, retrain promotion, deploy promoted model và export.
- Next.js lint/build cho frontend.
- Docker Compose config cho profile mặc định và profile `mlops`.
- Health check API/web sau khi build lại container.

5.5 Đánh giá định tính

UI Human Feedback sau chỉnh sửa giúp người dùng hiểu ngữ cảnh multi-speaker rõ hơn: transcript và speaker evidence được đặt cạnh speaker mapping, task evidence hiển thị đoạn hội thoại nguồn. Điều này giảm rủi ro gán nhầm người phụ trách khi audio có nhiều speaker.

Grafana/Prometheus phản ánh trạng thái worker, meeting pipeline và số liệu feedback. Các panel hiện tại đủ để quan sát pipeline trong phạm vi dự án; những chỉ số vận hành nâng cao như latency percentile, queue age hoặc GPU utilization có thể bổ sung khi mở rộng tải hoặc bổ sung GPU training.

Chapter 6

Thách thức và hạn chế

6.1 Diarization và speaker identity

Diarization chỉ tạo nhân speaker ẩn danh, không tự xác định danh tính thật của người nói. Vì vậy hệ thống cần bước human review để map `SPEAKER_00`, `SPEAKER_01` sang participant thật. Đây là giới hạn tự nhiên của diarization khi không có voice enrollment hoặc dữ liệu giọng nói tham chiếu.

6.2 Hiệu năng CPU

WhisperX, Pyannote và LLM inference đều tốn tài nguyên nếu chạy CPU. Môi trường local có thể xử lý chậm, đặc biệt ở lần đầu khi model cần tải về hoặc khởi tạo. Fine-tuning bằng QLoRA gần như cần GPU/CUDA nếu muốn có thời gian chạy hợp lý.

6.3 Fine-tuning phụ thuộc dữ liệu và GPU

Luồng fine-tune đã được chuẩn bị về code, Dockerfile, dependencies, queue và trigger. Tuy nhiên chất lượng fine-tuning phụ thuộc vào hai điều kiện: đủ dữ liệu correction chất lượng tốt và môi trường GPU phù hợp. Vì vậy báo cáo hiện tại xem fine-tuning là bước mở rộng của vòng lặp cải tiến, không dùng nó làm bằng chứng chính cho chất lượng baseline.

6.4 Thay đổi model được kiểm soát thủ công

Sau khi model candidate đạt điều kiện promotion, việc chuyển sang Ollama tag mới vẫn là thao tác manual/auditable. Cách làm này giúp mọi thay đổi model đều có dấu vết đánh giá, tránh trường hợp model mới thay thế baseline khi chưa được kiểm tra.

6.5 Giới hạn dữ liệu đánh giá

Smoke test chỉ có 5 samples và 13 action items. Bộ này đủ để bắt lỗi schema và regression đơn giản, nhưng chưa đủ để kết luận chất lượng tổng quát. Cần mở rộng tập đánh giá theo nhiều domain, ngôn ngữ, độ dài meeting và số speaker.

Chapter 7

Hướng phát triển

7.1 Mở rộng dữ liệu

Ưu tiên tiếp theo là thu thập thêm feedback thật từ UI, mở rộng gold set và bổ sung dữ liệu tiếng Việt/đa ngôn ngữ. Khi số correction đạt ngưỡng và có chất lượng ổn định, có thể chạy fine-tuning trên GPU rồi so sánh với baseline hiện tại bằng cùng một bộ đánh giá.

7.2 Nâng cấp evaluation

Hiện tại evaluation dựa trên matching action item ở mức mô tả. Hướng phát triển là thêm semantic matching, đánh giá riêng assignee/deadline/priority, đo hallucination rate và theo dõi confidence calibration nếu model hoặc pipeline có trả về điểm tin cậy.

7.3 Nâng cao vận hành

Các cải tiến vận hành không làm thay đổi pipeline cốt lõi nhưng giúp hệ thống bền vững hơn khi dữ liệu và tải xử lý tăng:

- Chuẩn hoá HTTPS/reverse proxy và secret manager.
- Backup/restore PostgreSQL và object storage cho artifacts.
- Role-based access control chi tiết hơn.
- Alert rules cho queue backlog, worker failure, API latency và GPU memory.
- Retention policy/cleanup cho raw artifacts.

7.4 Tự động hoá release có kiểm soát

Serving switch hiện là manual. Trong tương lai có thể thêm release policy bán tự động: model promoted được đưa vào canary, so sánh A/B trong một khoảng thời gian, sau đó mới chuyển traffic chính nếu không có regression.

Chapter 8

Kết luận

Dự án đã xây dựng được một hệ thống Meeting AI Agent end-to-end: upload audio, ASR, diarization, transcript theo lượt nói, trích xuất action items bằng LLM, human feedback, lưu trữ DB, monitoring và luồng LLMOps chuẩn bị cho fine-tuning. Các thay đổi gần đây giúp hệ thống có khả năng truy vết tốt hơn: feedback không còn là file rời rạc mà được lưu DB, trainer có queue riêng, retrain trigger có ngưỡng dữ liệu, promotion có audit và UI review phản ánh đúng bài toán multi-speaker.

Kết quả baseline Qwen2.5-3B đạt precision 0.767, recall 0.700 và schema failure 0.0% trên smoke test nhỏ. Điều này cho thấy pipeline đã ổn định về định dạng và luồng xử lý, nhưng chất lượng extraction vẫn cần cải thiện bằng dữ liệu feedback thật, mở rộng tập đánh giá và fine-tuning khi có đủ điều kiện.

Nhìn chung, dự án đáp ứng mục tiêu của một báo cáo Applied LLM/LLMOps: có bài toán NLP rõ ràng, pipeline LLM thực thi được, observability, human feedback, evaluation, retraining gate và quy trình thay đổi model có kiểm soát. Các phần tiếp theo nên tập trung vào chất lượng dữ liệu, đánh giá sâu hơn và nâng cao vận hành, thay vì xem chúng là thiếu hụt nền tảng của hệ thống hiện tại.

Tài liệu tham khảo

- [1] Microsoft Corporation. *Microsoft Work Trend Index: Hybrid Work Is Just Work*. Microsoft, 2022. <https://www.microsoft.com/en-us/worklab/work-trend-index>
- [2] M. Bain, J. Huh, T. Han, and A. Zisserman. WhisperX: Time-accurate speech transcription of long-form audio. *arXiv preprint arXiv:2303.00747*, 2022. <https://arxiv.org/abs/2303.00747>
- [3] H. Bredin et al. pyannote.audio 2.1 speaker diarization: Principle, benchmark, and recipe. In *Proceedings of Interspeech 2023*, 2023. <https://huggingface.co/pyannote/speaker-diarization-3.1>
- [4] Qwen Team. *Qwen2.5 Technical Report*. Alibaba Cloud, 2024. <https://arxiv.org/abs/2412.15115>
- [5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://arxiv.org/abs/2305.14314>
- [6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. <https://arxiv.org/abs/2106.09685>
- [7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023. <https://arxiv.org/abs/2212.04356>
- [8] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. <https://github.com/facebookresearch/faiss>
- [9] V. Shankar, R. Garcia, J. E. Gonzalez, and J. Hellerstein. Operationalizing machine learning: An interview study. *arXiv preprint arXiv:2209.09125*, 2022. <https://arxiv.org/abs/2209.09125>
- [10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. <https://arxiv.org/abs/2202.03629>

- [11] T. B. Brown et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. <https://arxiv.org/abs/2005.14165>
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Aiden, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. <https://arxiv.org/abs/1706.03762>
- [13] The Prometheus Authors. *Prometheus Monitoring System & Time Series Database*. <https://prometheus.io>, 2024.
- [14] Celery Project. *Celery: Distributed Task Queue*. <https://docs.celeryq.dev>, 2024.
- [15] S. Ramírez. *FastAPI Framework, high performance, easy to learn, fast to code*. <https://fastapi.tiangolo.com>, 2024.

Appendix A

Cấu trúc dự án hiện tại

```
1 web/                                # Next.js frontend
2 src/meeting_agent/
3   api/main.py                       # FastAPI endpoints
4   pipeline/
5     stt.py                          # WhisperX + diarization
6     worker_task.py                  # Celery tasks and queue
5     routing
7   mlops/
8     evaluate.py                     # Evaluation harness
9     finetune.py                     # GPU fine-tune
6     entrypoint
10    export.py                       # Feedback/data export
11    deploy.py                       # Controlled deploy flow
12 docker-compose.yml
13 Dockerfile.train
14 Makefile
15 docs/mlops-runbook.md
16 docs/source-audit-notes.md
17 data/eval/gold_smoke.jsonl
18 data/training/
```

Listing A.1: Các thư mục và file chính

Appendix B

Hướng dẫn chạy

B.1 Chạy môi trường mặc định

```
1 docker compose up -d --build
```

Listing B.1: Khởi động stack chính

Các URL thường dùng:

- Frontend: `http://localhost:3001`
- API health: `http://localhost:8000/health`
- Prometheus metrics: `http://localhost:8000/metrics`
- Grafana: port được cấu hình trong Docker Compose.

B.2 Chạy profile MLOps

```
1 make mlops-up
2 make retrain-check
3 make retrain-force
4 make train-image
```

Listing B.2: Khởi động các service MLOps

Profile này bổ sung Celery Beat, trainer worker và MLflow. Trainer consume queue `mlops`, tách khỏi worker xử lý meeting bình thường.

B.3 Deploy model promoted

```
1 make deploy-promoted-model APPLY=1
```

Listing B.3: Deploy model promoted có chủ đích

Deploy không tự động chạy ngay sau promotion. Operator phải thực hiện lệnh deploy để giữ audit trail và giảm rủi ro thay model ngoài ý muốn.

Appendix C

Ví dụ API

C.1 Upload meeting

```
1 curl -X POST http://localhost:8000/meetings \  
2   -F "file=@meeting.wav" \  
3   -F "participants=Alice,Bob,Charlie"
```

Listing C.1: Upload audio

C.2 Lấy trạng thái meeting

```
1 curl http://localhost:8000/meetings/{meeting_id}
```

Listing C.2: Xem trạng thái job

C.3 Gửi feedback

```
1 curl -X POST  
   http://localhost:8000/meetings/{meeting_id}/feedback \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4       "corrections": [  
5         {  
6           "task_id": "task-1",  
7           "field": "assignee",  
8           "old_value": "Bob",  
9           "new_value": "Alice"  
10        }  
11      ]  
12    }'
```

Listing C.3: Gửi correction từ human review

Appendix D

Cấu hình môi trường quan trọng

Biến	Mục đích
DATABASE_URL	Kết nối PostgreSQL
REDIS_URL	Redis broker/cache
OLLAMA_BASE_URL	Endpoint Ollama
OLLAMA_LLM_MODEL	Model LLM đang phục vụ
WHISPER_MODEL	Model WhisperX
WHISPER_DEVICE	cpu hoặc cuda
HF_TOKEN	Token cho model diarization nếu cần
RETRAIN_MIN_CORRECTIONS	Ngưỡng correction để retrain check
MLFLOW_TRACKING_URI	Tracking URI cho MLflow

Các giá trị bí mật phải đặt trong `.env`; `.env.example` chỉ giữ tên biến và giá trị mẫu không nhạy cảm.